

Instituto Superior de Engenharia do Porto
Politécnico do Porto

Licenciatura em Engenharia Electrotécnica e de Computadores
Sistemas Computacionais Avançados (SISTCA) / Advanced
Computing Systems

Lab Classes Script: Hand Tracking Using Computer Vision

Version Control

Version number	Date issued	Authors	Update information
V1.0	April 2026	Rodrigo Pires (1210697) Miguel Veloso (1212013) Rui Rodrigues (1221909) Maria Inês (1211346) Supervision: Prof. Rui Nuno Castro	Original version of the script.

Contents

1	Introduction	3
1.1	Scientific/Technological Context	3
1.2	Motivation for this topic/lab script	3
1.3	Objectives (of this lab script)	3
1.4	Structure (of this lab script)	3
2	Theoretical (scientific/technological background)	5
2.1	Main Theoretical Concepts / Terminologies	5
2.1.1	Computer Vision	5
2.1.2	Color Space	5
2.1.3	Inference	6
2.1.4	Landmarks	6
2.2	State-of-the-Art	7
2.2.1	VSCode	7
2.2.2	Python	7
2.2.3	OpenCV	7
2.2.4	MediaPipe Model	7
3	Outlook of the technology	8
3.1	Setup/Installation (SW/HW)	8
3.1.1	Windows Setup	8
3.1.2	Linux Setup	9
3.2	First Steps	9
4	Tutorial/Functionality	12
4.1	Algorithmic Rationale	12
4.2	Environment Setup and Core Libraries	12
4.3	Video Capture and Landmark Mapping	13
4.4	Image Pre-processing: The "Why" and "How"	13
4.5	Kinematic Logic: The Coordinate System	13
4.6	Visual Output and Resource Deallocation	14
4.7	Advanced Integration: Custom AI Model and Mouse Control (HCI)	15
4.7.1	Feature Engineering (Wrist Relativity)	15
4.7.2	Training the Classifier	15
4.7.3	Real-Time Mouse Control (HCI)	16
5	Exercise(s)	17
5.1	Problem A - Alt Tab Command	17
5.1.1	Resolution A	17
5.2	Problem B - Screenshot Automation	17
5.2.1	Resolution B	17
6	Challenge: The "Virtual Zoom"	19
	Annexes	21

1 Introduction

1.1 Scientific/Technological Context

In the rapidly evolving landscape of technology and artificial intelligence, Computer Vision has transitioned from a niche research area to a cornerstone of modern applications, namely Human-Computer Interaction (HCI). At its core lies Image Processing, a fundamental technique that enables computers to interpret and manipulate meaningful data from visual inputs.

The technology to bridge the gap between human intent and machine execution through real-time hand gesture recognition is increasingly relevant in various domains, such as sterile medical environments, immersive augmented reality and assistive tools for individuals with motor impairments. By moving beyond traditional input devices we can create more intuitive and natural interfaces that enhance user experience and accessibility.

Technologically this lab script centers on the synergy between OpenCV, a powerful open-source computer vision library and Google's MediaPipe, a framework for building multimodal applied machine learning pipelines, specifically its hand tracking solution that utilizes a pre-trained palm detector and a hand landmark model to provide a 21-point 3D hand skeletal map.

1.2 Motivation for this topic/lab script

In large part due to the COVID-19 pandemic, the world has witnessed a significant shift towards remote interactions and contactless technologies. This has accelerated the demand for more intuitive and natural interfaces that can bridge the gap between humans and machines.

Coupled with the advancements in machine learning and the availability of powerful libraries such as OpenCV and MediaPipe, tackling the challenge of real-time hand gesture recognition seemed like a natural choice for a lab script in the context of Advanced Computing Systems (SISTCA).

1.3 Objectives (of this lab script)

The primary objective of this lab script is the development of a distributed real-time hand gesture recognition system. This document defines both the theoretical concepts to be acquired and the practical milestones to be achieved.

Upon completion, students should be able to:

- **Learn (Theoretical Concepts):** Understand the principles of edge computing, spatial anatomical feature extraction (landmarks), and the fundamentals of lightweight Local Area Network (LAN) communication through socket protocols.
- **Do (Practice and Milestones):**
 - Configure a Python development environment.
 - Capture video streams and extract hand topology using the MediaPipe library.
 - Implement geometric logic to identify and count raised fingers.
 - Develop socket communication to transmit the gesture classification to another machine on the network with minimum latency.

1.4 Structure (of this lab script)

The remainder of this document is organized as follows:

- Section 2 (Theoretical): Presents the fundamental theoretical concepts and the state-of-the-art of the addressed technologies.

- Section 3 (Outlook of the technology): Describes the hardware and software configuration requirements to start the project.
- Section 4 (Tutorial/Functionality): Constitutes the core of this lab script, guiding the student through a step-by-step tutorial to build the system.
- Section 5 (Exercise(s)): Proposes practical exercises and provides their respective resolutions.
- Section 6 (Challenge): Presents an autonomous challenge for the students to consolidate the learned concepts.
- Section 7 (References): Lists all the bibliographic references used throughout the document.

2 Theoretical (scientific/technological background)

2.1 Main Theoretical Concepts / Terminologies

2.1.1 Computer Vision

Computer Vision (CV) is a field of Artificial Intelligence that enables computers to see and interpret the real world. In any given image, the machine will only receive a NumPy Array containing values (0-255) that represent the light intensity of each pixel. Without a way to understand the information, it can't perceive objects the way humans can. However, by providing it with structured data, algorithms can be trained to identify color and pixel patterns, allowing them to accurately classify objects [1].

2.1.2 Color Space

A Color Space is a mathematical model that defines how the colors are represented numerically in an image. Each pixel stores a value or a set of values that instruct the screen on what light intensities to emit; depending on the model, these numerical values will represent very different colors.

RGB (Red Green Blue) - The Default Format used by most Deep Learning Models and the standard for every Phone or TV screen, Monitor or LED. Each pixel consists of 3 smaller sub-pixels: Red, Green, and Blue. By varying the light intensity of each sub-pixel (0-255), the system can display any given color [2].

Red (R)	Green (G)	Blue (B)	Color Name
0	0	0	Black
255	255	255	White
255	0	0	Red
0	255	0	Green
0	0	255	Blue
0	255	255	Cyan
255	0	255	Magenta
255	128	0	Orange

Table 1: Examples of RGB Color Values

BGR (Blue Green Red) - The Default Format used by the OpenCV Library, due to it being developed in 1999 when the BGR color format was the standard for software development and camera manufacturers [3]. In essence, it is similar to the RGB Format, with the notable difference of the Red and Blue channels swapped. In order to implement a Deep Learning Model with OpenCV, it's required to manually convert the image to the standard RGB format.

HSV (Hue Saturation Value) - While RGB is what hardware uses to display color, the HSV format aligns more closely to how people perceive color. It consists of 3 different values:

- **Hue** - The base color represented on a 360° color wheel.
- **Saturation** - Intensity of the color, from vibrant to faded.
- **Value (Brightness)** - Amount of light hitting the color.

Because the Hue component remains consistent regardless of lighting variations, this format is ideal for projects that require object tracking and color segmentation [2].

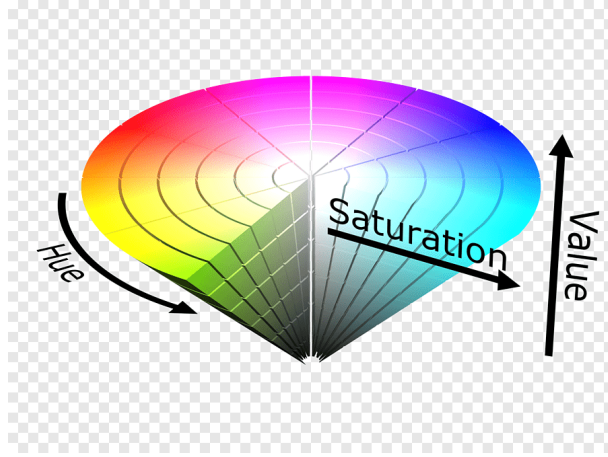


Figure 1: HSV Color Pattern

Grayscale - Grayscale is a single-channel image format that removes all hue and saturation, leaving only its brightness value. By reducing the three channels down to only one [3], it greatly decreases the processing load. Its increased simplicity enables the creation of color masks that can filter out backgrounds and simplify complex objects for easier analysis; this is a fundamental pre-processing step for Algorithm and Deep Learning training.

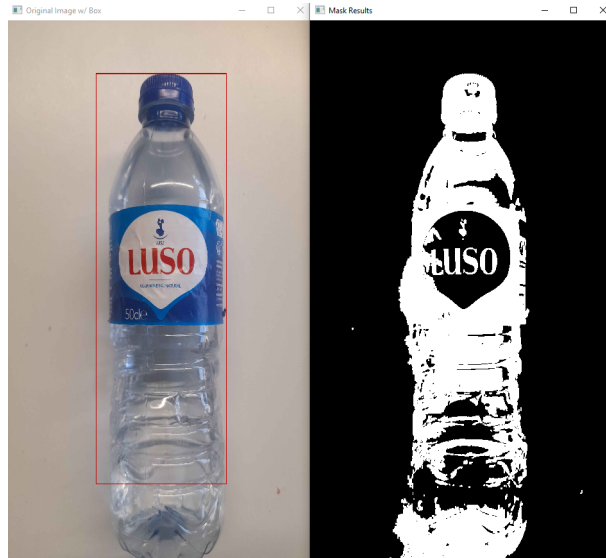


Figure 2: Color Mask Example using Grayscale

2.1.3 Inference

Inference is the process where an AI Algorithm makes predictions based on the data provided. In this project, the data consists of the live feed being captured by the camera, which is continuously being fed into the model for analysis. Due to the nature of this software, the inference must be done with relatively high speed and precision to ensure low latency and smooth operation.

2.1.4 Landmarks

The last step of the analysis is to show its results. Once Inference is done correctly, the model will return 21 sets of coordinates (x,y,z). These are landmarks spread throughout the

hand, from the wrist to the fingertips. The software then reconstructs the hand gesture by connecting these coordinates together, creating a uniform pattern that will allow us to use a model and recognize what motion or gesture the hand is making.

2.2 State-of-the-Art

2.2.1 VSCode

VSCode is a lightweight but versatile Integrated Development Environment (IDE) with support for hundreds of languages. Not only is it easy to use, but it provides various tools for computer vision development such as the integrated terminal for package management (pip), and GIT integration for work synchronization between project members. VSCode also supports the LaTeX Language, which allows us to write this script and compile it into a PDF file without resorting to online LaTeX Editors with paid subscriptions and limited compile times [a].

2.2.2 Python

Python is one of the most widely used programming languages, known for its versatility, fast prototyping, and ease-of-use. It was chosen for having better integration with OpenCV and MediaPipe, and being able to perform complex mathematical operations while maintaining fast performance, which is required when working with real-time data [b].

2.2.3 OpenCV

OpenCV is an Open Source Computer Vision Library that acts as the eyes for this project, as well as the bridge between the camera feed and the software. It is responsible for the frame acquisition, its pre-processing, the drawing of the model inference results, as well as displaying the software's logic directly onto the video feed. It's a versatile tool that works efficiently with other AI Models for a wide range of computer vision projects, from object detection to face recognition, and it was chosen for its ease-of-use, optimized real-time performance, and image processing [4].

2.2.4 MediaPipe Model

MediaPipe is an open source framework developed by Google; it specializes in real-time, machine learning solutions for live video, audio, and streaming [5]. For this project, it uses a palm detector and a landmark model to draw 21 points throughout the image of a hand with very low latency [6], which enables it to work in tandem with the camera and OpenCV to continuously analyze the images and display the results at very high speeds.

3 Outlook of the technology

3.1 Setup/Installation (SW/HW)

3.1.1 Windows Setup

To start the lab script, students must ensure their Windows workstations meet the following requirements:

Hardware Requirements:

- A personal computer (desktop or laptop).
- A video camera (built-in or USB peripheral).
- An active connection to a Local Area Network (LAN), either via Wi-Fi or an Ethernet cable.

Software and Development Platform Requirements:

- Operating System: Microsoft Windows (Windows 10 or 11 is recommended).
- Platform: Python 3.8 (or higher) installed. During the installation process, it is strictly necessary to check the "Add Python to PATH" option to allow the execution of global commands, as shown in Figure 3.

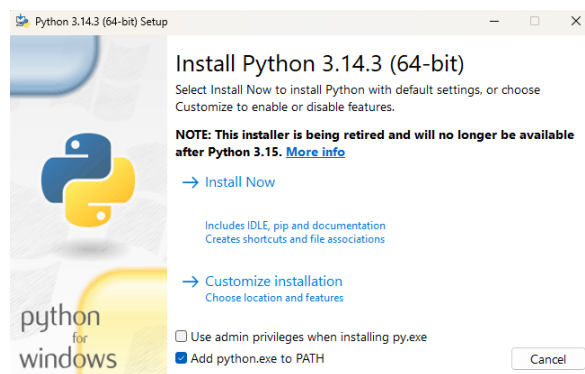


Figure 3: Adding Python to PATH during installation.

- Environment: An Integrated Development Environment (IDE) such as Visual Studio Code (VS Code) is recommended.
- Dependencies: The computer vision libraries must be installed via the Windows Command Prompt (cmd) or PowerShell. The student must execute the following command using the pip package manager, as demonstrated in Figure 4:

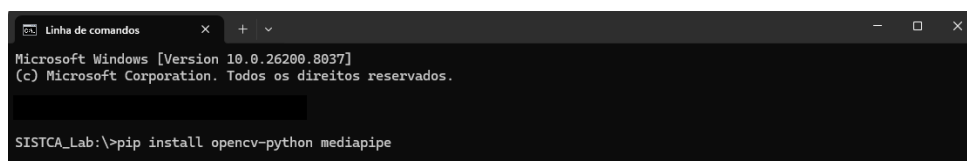


Figure 4: Installing required libraries via Command Prompt.

3.1.2 Linux Setup

If you are using Linux we'll assume you have a basic understanding of the command line and already have Visual Studio Code installed, so you can follow these steps to set up your environment:

1. Choose or create the directory where you want to create your project.
2. Open a terminal and make sure your system is up to date by running:

```
# Linux Update Commands
sudo apt update
sudo apt upgrade
```

3. Install Python and pip if you haven't already:

```
# Linux Python Installation Commands
sudo apt install python3 python3-pip python3-venv
```

4. Inside your project directory, open a terminal and create a virtual environment for your project (optional but recommended):

```
# Linux Virtual Environment Commands
python3 -m venv myvenv
source myvenv/bin/activate
```

5. Inside Visual Studio Code (*in your project directory*), open the Command Palette (Ctrl+Shift+P) and select "Python: Select Interpreter". Choose the interpreter from your virtual environment (it should be something like **'myvenv/bin/python'**).
6. Upgrade pip and install the following packages:

```
# Linux Package Installation Commands
pip install --upgrade pip
pip install opencv-python mediapipe scikit-learn pandas
```

3.2 First Steps

The development of the Hand-Tracking Software begins with the use of the **OpenCV** library alongside the front-facing camera to obtain a video feed in real-time, aiming to continuously capture hand movements and gestures.

The function `cap = cv2.VideoCapture(n)` is responsible for the camera initialization. The index $n = 0$ refers to built-in webcams commonly found in laptops, whereas higher indexes ($n = 1, 2, \dots$) allow access to secondary cameras connected via USB. It's crucial for both the camera and processor to capture the images at a high framerate to ensure smooth hand tracking and increased precision for analysis.

Before sending the image to the algorithm, pre-processing is required. OpenCV captures frames in the **BGR** format (Blue, Green, Red). The function `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)` will convert it to the **RGB** format (Red, Green, Blue), which is the standard format used by most Deep Learning Models to ensure correct Inference (AI Processing). In addition, mirroring the frame with the function `cv2.flip(frame, 1)` provides a more intuitive feed for the user.

```
import cv2

cap = cv2.VideoCapture(0)
cv2.namedWindow('HandTracker', cv2.WINDOW_NORMAL)
```

```

while True:
    ret, frame = cap.read()
    if not ret:
        print("Error: Failed to grab frame.")
        break

    frame = cv2.flip(frame, 1)
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    cv2.imshow('Hand Tracker', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

```

Listing 1: Basic Camera Feed using OpenCV

The next step is to pass the frames through the MediaPipe model, which analyses the image and returns 21 Hand Landmarks placed throughout the hand, storing their respective coordinates. The analysis is triggered by calling the function `results = hands.process(rgb_frame)`. Following the Inference, if a hand is detected, the coordinates are connected by lines and drawn onto the original frame, which is then displayed in the window feed.

```

import cv2
import mediapipe as mp

# MediaPipe Model Config
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils
hands = mp_hands.Hands(min_detection_confidence=0.9,
                        min_tracking_confidence=0.9)

# MediaPipe Coordinates to Finger Tip ID (Index, Middle, Ring, Pinky)
finger_tips = [8, 12, 16, 20]
thumb_tip = 4

# OpenCV Camera Feed
cap = cv2.VideoCapture(0)

while cap.isOpened():
    ret, frame = cap.read()
    if not ret: break

    frame = cv2.flip(frame, 1)
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = hands.process(rgb_frame)

    if results.multi_hand_landmarks:
        # We use 'enumerate' to know which hand we're looking at
        for hand_idx, hand_landmarks in enumerate(results.
            multi_hand_landmarks):

            # Draw Lines and Coordinates in Hand
            mp_drawing.draw_landmarks(frame, hand_landmarks, mp_hands.
                HAND_CONNECTIONS)

            # Find which hand (left or right) is being detected

```

```

hand_label = results.multi_handedness[hand_idx].
    classification[0].label
raised_fingers = 0

# 1. Thumb Logic (X-Axis)
if hand_label == "Right":
    if hand_landmarks.landmark[thumb_tip].x < hand_landmarks.
        landmark[thumb_tip - 1].x:
        raised_fingers += 1
else: # "Left"
    if hand_landmarks.landmark[thumb_tip].x > hand_landmarks.
        landmark[thumb_tip - 1].x:
        raised_fingers += 1

# 2. Other 4 Fingers Logic (Y-Axis)
for tip in finger_tips:
    # On the Window, the Y-Axis increases downward
    if hand_landmarks.landmark[tip].y < hand_landmarks.
        landmark[tip - 2].y:
        raised_fingers += 1

# Write Text in Window
cv2.putText(frame, f'{hand_label}_hand: {raised_fingers}_
    fingers',
        (10, 50 + (hand_idx * 50)),
        cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.
            LINE_AA)

cv2.imshow('Hand_Tracker', frame)
if cv2.waitKey(1) & 0xFF == ord('q'): break

cap.release()
cv2.destroyAllWindows()

```

Listing 2: Mediapipe Frame Analysis

This continuous cycle of frame acquisition and immediate algorithmic analysis makes up the fundamental core of this project. The following chapter will elaborate on how the 21 hand landmarks can be used to detect patterns and movements. By leveraging this data, we can send commands to the computer, enabling control over the system through hand gestures alone.

4 Tutorial/Functionality

In this chapter, we will dissect the codebase developed for real-time hand detection and finger counting. Beyond simply presenting the code, this section elucidates the underlying algorithmic thought process and details the fundamental commands provided by the **OpenCV** and **MediaPipe** libraries.

4.1 Algorithmic Rationale

Before delving into the syntactic implementation, it is crucial to understand the logical pipeline of the system:

1. **Data Acquisition:** Capture a continuous stream of visual data from a hardware peripheral (webcam).
2. **Normalization:** Mirror the image for intuitive user interaction and convert the color space to meet the machine learning model's prerequisites.
3. **Inference:** Pass the normalized image through the MediaPipe neural network to extract a topological graph of the hand (landmarks).
4. **Kinematic Evaluation:** Utilize the spatial coordinates of these landmarks to deduce the state of each digit (extended or folded) based on anatomical constraints.
5. **Feedback:** Overlay the computed state onto the original image stream to provide real-time visual feedback.

4.2 Environment Setup and Core Libraries

We initiate the script by importing the requisite libraries and defining the core MediaPipe objects.

```
import os
os.environ["QT_QPA_PLATFORM"] = "xcb"

import cv2
import mediapipe as mp

mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils
hands = mp_hands.Hands(min_detection_confidence=0.5,
                        min_tracking_confidence=0.5)
```

Basic Commands Explained:

- `import cv2`: Imports the OpenCV library, the industry standard for real-time computer vision and image processing tasks.
- `mp.solutions.hands`: Accesses the specific hand tracking module within the broader MediaPipe framework.
- `mp_hands.Hands(...)`: Instantiates the machine learning model. The parameters `min_detection_confidence` and `min_tracking_confidence` act as statistical thresholds. Setting them to 0.5 dictates that the model must be at least 50% certain that a hand is present to begin or continue tracking it, balancing processing speed with accuracy.

4.3 Video Capture and Landmark Mapping

The subsequent phase involves initializing the hardware interface and establishing the indices for the finger landmarks.

```
cap = cv2.VideoCapture(1)
cv2.namedWindow('Hand Tracker', cv2.WINDOW_NORMAL)

# IDs for the fingertip landmarks in MediaPipe (Index, Middle, Ring,
#   Pinky)
pontas_dos_dedos = [8, 12, 16, 20]
ponta_do_polegar = 4
```

Basic Commands Explained:

- `cv2.VideoCapture(1)`: Creates a video capture object. The argument 1 targets the secondary camera interface, while 0 typically targets the primary built-in webcam.
- `cv2.namedWindow(...)`: Initializes a graphical user interface (GUI) window. The `WINDOW_NORMAL` flag allows the user to resize the output window dynamically.

4.4 Image Pre-processing: The "Why" and "How"

Within the continuous execution loop (`while True`), raw frames are acquired and mathematically transformed.

```
while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame = cv2.flip(frame, 1)
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = hands.process(rgb_frame)
```

Thought Process & Basic Commands:

- `cap.read()`: Extracts the subsequent frame from the video stream. It returns a boolean (`ret`) indicating success, and the matrix of pixels (`frame`).
- `cv2.flip(frame, 1)`: Applies a horizontal mirror reflection across the vertical axis (indicated by the 1). Without this operation, moving your hand to the left would result in the on-screen hand moving to the right, which causes severe cognitive dissonance for the user.
- `cv2.cvtColor(...)`: OpenCV natively captures color arrays in Blue-Green-Red (BGR) format. However, the MediaPipe neural network was trained on Red-Green-Blue (RGB) datasets. This function strictly translates the color channels to ensure the model processes the image accurately.
- `hands.process(rgb_frame)`: Executes the core inference step, passing the RGB matrix into the neural network and returning a `results` object containing the spatial coordinates of any detected hands.

4.5 Kinematic Logic: The Coordinate System

When hands are detected, the algorithm iterates through them and evaluates the spatial configuration of the digits.

```

if results.multi_hand_landmarks:
    for hand_idx, hand_landmarks in enumerate(results.
        multi_hand_landmarks):
        mp_drawing.draw_landmarks(frame, hand_landmarks, mp_hands.
            HAND_CONNECTIONS)

        hand_label = results.multi_handedness[hand_idx].
            classification[0].label
        dedos_levantados = 0

        # 1. Thumb Logic (X-Axis)
        if hand_label == "Right":
            if hand_landmarks.landmark[ponta_do_polegar].x <
                hand_landmarks.landmark[ponta_do_polegar - 1].x:
                dedos_levantados += 1
        else: # "Left"
            if hand_landmarks.landmark[ponta_do_polegar].x >
                hand_landmarks.landmark[ponta_do_polegar - 1].x:
                dedos_levantados += 1

        # 2. Logic for the remaining 4 fingers (Y-Axis)
        for ponta in pontas_dos_dedos:
            if hand_landmarks.landmark[ponta].y < hand_landmarks.
                landmark[ponta - 2].y:
                dedos_levantados += 1

```

Thought Process for Finger Counting: The algorithm's core logic is predicated on the digital image coordinate system, where the origin (0,0) is located at the top-left corner of the image. Consequently, moving downwards increases the Y-value, and moving rightwards increases the X-value.

- **The Four Fingers (Index to Pinky):** To determine if a finger is "up", we compare the Y-coordinate of the fingertip (`landmark[ponta]`) to the Y-coordinate of the knuckle two joints below it (`landmark[ponta - 2]`). Due to the inverted Y-axis, a numerically smaller Y-value indicates a higher physical position. Thus, if $Y_{tip} < Y_{knuckle}$, the finger is extended.
- **The Thumb Anomaly:** The thumb operates on a different biomechanical plane, rotating horizontally rather than folding vertically. Therefore, we evaluate the X-axis. A right thumb pointing leftwards (towards the center of the body) has a smaller X-value than its base joint. This logic must be strictly inverted for the left hand, necessitating the `hand_label` conditional check.

4.6 Visual Output and Resource Deallocation

Finally, the extracted data is overlaid onto the original BGR frame for the user's viewing.

```

cv2.putText(frame, f'{hand_label} hand: {dedos_levantados}
    fingers', (10, 50 + (hand_idx * 50)),
        cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2, cv2.
            LINE_AA)

cv2.imshow('Hand Tracker', frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

```

```
cap.release()
cv2.destroyAllWindows()
```

Basic Commands Explained:

- `cv2.putText(...)`: Renders text directly onto the pixel matrix. The algorithmic offset `50 + (hand_idx * 50)` ensures that if multiple hands are detected, their respective text labels stack vertically without visual occlusion.
- `cv2.imshow(...)`: Commands the operating system to refresh the GUI window with the newly processed frame.
- `cv2.waitKey(1)`: Halts execution for 1 millisecond to listen for keyboard interrupts. If the user presses the 'q' key, the `while` loop breaks.
- `cap.release()` & `cv2.destroyAllWindows()`: These cleanup functions are crucial; they sever the connection to the hardware camera to prevent system locks and safely close all active graphical windows.

4.7 Advanced Integration: Custom AI Model and Mouse Control (HCI)

While in Section 4.5 successfully count fingers, they are highly dependent on 2D perspective and camera angles. To make the system truly robust, we can replace these hardcoded rules with a custom Machine Learning (ML) model and utilize the predictions for Human-Computer Interaction (HCI), specifically controlling the operating system's mouse pointer.

4.7.1 Feature Engineering (Wrist Relativity)

To train a robust model, the input data must be position-invariant. If raw screen coordinates are fed into the model, the algorithm will overfit to the hand's pixel location on the screen rather than learning its structural shape.

We set the wrist (Landmark 0) as the origin point (0,0,0). All other 20 landmarks are calculated relative to the wrist using the following transformations:

$$x_{rel} = x_i - x_{wrist} \quad (1)$$

$$y_{rel} = y_i - y_{wrist} \quad (2)$$

$$z_{rel} = z_i - z_{wrist} \quad (3)$$

Listing 3: Extracting 3D Position-Invariant Features

```
landmark_data = []
wrist = hand_landmarks.landmark[0]

for landmark in hand_landmarks.landmark:
    rel_x = landmark.x - wrist.x
    rel_y = landmark.y - wrist.y
    rel_z = landmark.z - wrist.z
    landmark_data.extend([rel_x, rel_y, rel_z])
```

4.7.2 Training the Classifier

By capturing a dataset containing roughly 100-200 variations of each gesture (0 to 5 fingers) using these engineered relative coordinates, the data becomes tabular. This allows us to utilize the `scikit-learn` library to train a Random Forest Classifier.

Listing 4: Training a Random Forest Model

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
import pickle

# Load dataset and split into 80% Training and 20% Testing
df = pd.read_csv('hand_data.csv')
X = df.drop('label', axis=1)
y = df['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Train the model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Save the trained model
with open('hand_model_3d.pkl', 'wb') as f:
    pickle.dump(model, f)
```

4.7.3 Real-Time Mouse Control (HCI)

With a trained model (.pkl file) capable of recognizing gestures, we can import the `pyautogui` library to bridge the gap between Python and the operating system.

The tip of the index finger (Landmark 8) dictates the cursor's spatial X and Y coordinates, mapped to the monitor's resolution. When the ML model classifies the hand structure as a closed fist ("0"), the script triggers a left-click event.

Listing 5: HCI Mouse Control Integration

```
import pyautogui
import time

# Mouse mapping logic using the Index Finger
index_finger_tip = hand_landmarks.landmark[8]
mouse_x = int(index_finger_tip.x * screen_w)
mouse_y = int(index_finger_tip.y * screen_h)
pyautogui.moveTo(mouse_x, mouse_y, _pause=False)

# Clicking logic based on ML prediction
if prediction == '0':
    if (current_time - last_click_time) > 1.0: # 1-second cooldown
        pyautogui.click()
        last_click_time = current_time
```


5 Exercise(s)

5.1 Problem A - Alt Tab Command

In this setting of using our hands to control our computer, the ability to "alt-tab" anytime is a very useful feature that allows the user to navigate the entire workspace in an instant. For this exercise, we want to map this command to a finger or a gesture, so that we can still use the index finger to control the mouse cursor around the screen.

5.1.1 Resolution A

Using the Ring Finger (Landmark 16), we can create a simple geometric rule to trigger the Alt-Tab keyboard macro using PyAutoGUI.

```
rf_point = hand_landmarks.landmark[16] # Ring Finger Tip
rf_base = hand_landmarks.landmark[14]  # Ring Finger PIP Joint (Base)

# If Point is Higher than Base, Ring Finger is raised: Activate Command
if rf_point.y < rf_base.y:
    # Flag controls Alt key press status, so it doesnt conflict with
    # normal computer use
    if not f_alttab:
        pyautogui.keyDown('alt')      # Presses Alt key, keeps it
        pressed
        pyautogui.press('tab')        # Presses Tab key once to enter
        menu
        f_alttab = True                # Activates Flag
    else:
        if f_alttab:
            pyautogui.keyUp('alt')     # Releases Alt key if it was
            pressed
            f_alttab = False           # Deactivates Flag
```

Listing 6: Exercise A - Alt Tab Implementation

5.2 Problem B - Screenshot Automation

Enhance the existing real-time hand-tracking and mouse-control software to trigger a screenshot when a specific gesture is detected[cite: 416]. The system needs to recognize when the user fully extends their pinky finger and then execute the **Windows + Shift + S** keyboard shortcut[cite: 417]. Importantly, the program must continue running right after pressing the shortcut[cite: 418]. This allows the user to use the existing hand gestures to move the mouse and select the area for the screenshot[cite: 419].

5.2.1 Resolution B

To add this feature, we use the `pyautogui` library to simulate keyboard presses[cite: 420]. The logic depends on checking the position of the pinky finger using the MediaPipe hand tracking model[cite: 421].

According to the model, the tip of the pinky finger is point (landmark) 20[cite: 422]. We can figure out if the finger is raised by comparing the Y-coordinate of the tip (landmark 20) with a lower joint, like landmark 18[cite: 422]. In images, the Y-axis goes from top to bottom, meaning a smaller Y-value is physically higher on the screen[cite: 422]. So, if the tip's Y-value is smaller than the joint's Y-value, the finger is raised[cite: 423].

A major challenge is preventing the shortcut from triggering multiple times a second without pausing the video feed[cite: 424]. If we use a simple `time.sleep()` function, the whole program

freezes[cite: 425]. This would stop the camera and the mouse control, making it impossible to crop the screenshot[cite: 426]. Instead, we use a non-blocking timer with `time.time()`[cite: 427]. This method checks the current time and ensures the shortcut is only pressed once every few seconds[cite: 427]. Because the main loop never stops, the user can immediately close their hand (which our Custom AI model predicts as state 0) to click and drag the mouse to capture the screen[cite: 428].

The code modification required to implement this logic within the existing software is shown below[cite: 429]:

```
import pyautogui
import time

# --- Setup Configuration (Place before the main while loop) ---
pyautogui.PAUSE = 0
last_action_time = 0
COOLDOWN_SECONDS = 2.0

# ... [Inside the 'for hand_landmarks in results.multi_hand_landmarks:'
#      loop] ...

# 1. Check if the pinky finger is raised
pinky_tip = hand_landmarks.landmark[20].y
pinky_pip = hand_landmarks.landmark[18].y
is_pinky_raised = pinky_tip < pinky_pip

current_time = time.time()

# 2. Trigger the shortcut with a non-blocking timer
if is_pinky_raised and (current_time - last_action_time >
    COOLDOWN_SECONDS):
    pyautogui.hotkey('win', 'shift', 's')
    last_action_time = current_time # Reset the cooldown timer

# ... [The rest of the existing gesture control logic remains unchanged]
# ...
```

Listing 7: Exercise B - Screenshot Automation Implementation

This approach allows the software to take a screenshot and immediately transition back to gesture control, fulfilling the exercise requirements while keeping the application smooth and responsive[cite: 430, 431].

6 Challenge: The "Virtual Zoom"

To conclude this lab script, the final challenge consists of expanding the system's capabilities, focusing on accessibility and document navigation.

Problem Statement: When analyzing complex electrical schematics or reading technical documentation, it is often necessary to zoom in or out without taking your hands off the task being performed. The objective is to modify the `run_ai.py` script to function as a gesture-controlled magnification tool.

Requirements and Objectives:

- **Movement 1 and Command 1 (Zoom In):** Detect when the user's hand is fully open (5 fingers raised). This movement must trigger the `Ctrl +` shortcut via the `pyautogui.hotkey('ctrl', '+')` command.
- **Movement 2 and Command 2 (Zoom Out):** Detect when the user's hand is fully closed (0 fingers raised). This movement must trigger the `Ctrl -` shortcut via the `pyautogui.hotkey('ctrl', '-')` command.
- **Cooldown Implementation:** As learned in Exercise B, you must implement a non-blocking timer using `time.time()` to prevent the command from being triggered multiple times per second. A cooldown interval of at least 1.2 seconds is recommended.

Implementation Hint: You can solve this challenge by checking your Machine Learning model's output variable (`prediction`). Confirm if the string value is exactly `'5'` or `'0'` before executing the respective shortcut command and resetting the non-blocking timer.

References

- [1] S. Kumari, A. Kushwaha, S. Singh, and R. SAIKOMAL, “Hand gesture recognizer,” *International Journal of Science and Advanced Technology*, vol. 16, Oct 2025.
- [2] S. Mopidevi, “Hand gesture recognition and voice conversion for deaf and dumb,” *E3S Web of Conferences*, vol. 391, p. 01060, Jan 2023.
- [3] H. Deep, M. K. S, P. Verma, and K. Chinmay, “Intellimotion: Real-time hand gesture and pose tracking using opencv and mediapipe,” in *IEEE Conference*, June 2025.
- [4] S. Biswas, A. Nandy, A. K. Naskar, and R. Saw, “Mediapipe with lstm architecture for real-time hand gesture recognition,” in *Communications in Computer and Information Science*, Springer, Jan 2024.
- [5] SAS, “Computer vision: What it is and why it matters.” https://www.sas.com/pt_pt/insights/analytics/computer-vision.html. Accessed: 2026.
- [6] Roboflow, “Opencv color spaces.” <https://blog.roboflow.com/opencv-color-spaces/>. Accessed: 2026.
- [7] S. Mallick, “Why does opencv use bgr color format?.” <https://learnopencv.com/why-does-opencv-use-bgr-color-format/>. Accessed: 2026.
- [8] OpenCV, “About opencv.” <https://opencv.org/about/>. Accessed: 2026.
- [9] Google, “Mediapipe.” <https://mediapipe.org/>. Accessed: 2026.
- [10] Google, “Mediapipe hands solutions.” <https://mediapipe.readthedocs.io/en/latest/solutions/hands.html>. Accessed: 2026.
- [11] Google, “Mediapipe.” <https://mediapipe.org/>. Accessed: 2026.
- [12] Google, “Mediapipe hands solutions.” <https://mediapipe.readthedocs.io/en/latest/solutions/hands.html>. Accessed: 2026.
- [13] PyAutoGUI, “Pyautogui documentation.” <https://pyautogui.readthedocs.io/en/latest/mouse.html>. Accessed: 2026.
- [14] Microsoft, “Visual studio code.” <https://code.visualstudio.com/>. Accessed: 2026.
- [15] Python Software Foundation, “About python.” <https://www.python.org/about/>. Accessed: 2026.

Annexes